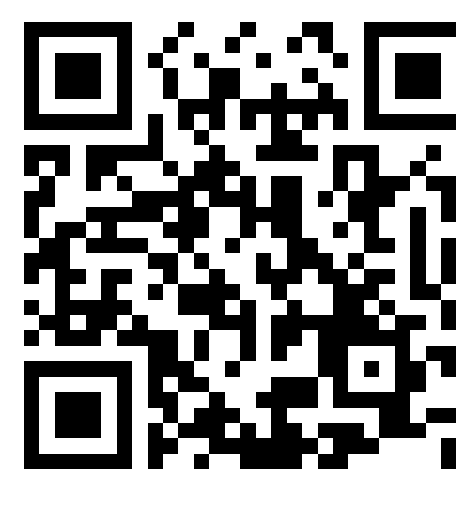




Introduction

- Advances in AI have boosted GPU adoption. E.g., all following apps involve frequent GPU-Storage data movement during execution:
 - Scientific apps** use GPUs for **data analysis and processing**
 - Training apps** load large datasets onto GPU and write checkpoints to storage
 - Inference engines** usually load models onto GPU and transfer intermediate data during inference serving (e.g., KV cache data in LLMs)
- Traditional GPU-Storage data transfer uses CPU/DRAM as a bridge
 - Limiting I/O scalabilities due to the number of CPU cores
 - Increasing host memory overhead, latency, and CPU utilization
- New technologies (e.g., GPUDirect Storage and NIXL library) allow direct GPU-Storage data transfer, bypassing CPU bounce buffer
- However, **limited research assessing the practical benefits of GDS and NIXL**, and their associated trade-offs remain unexplored

Background

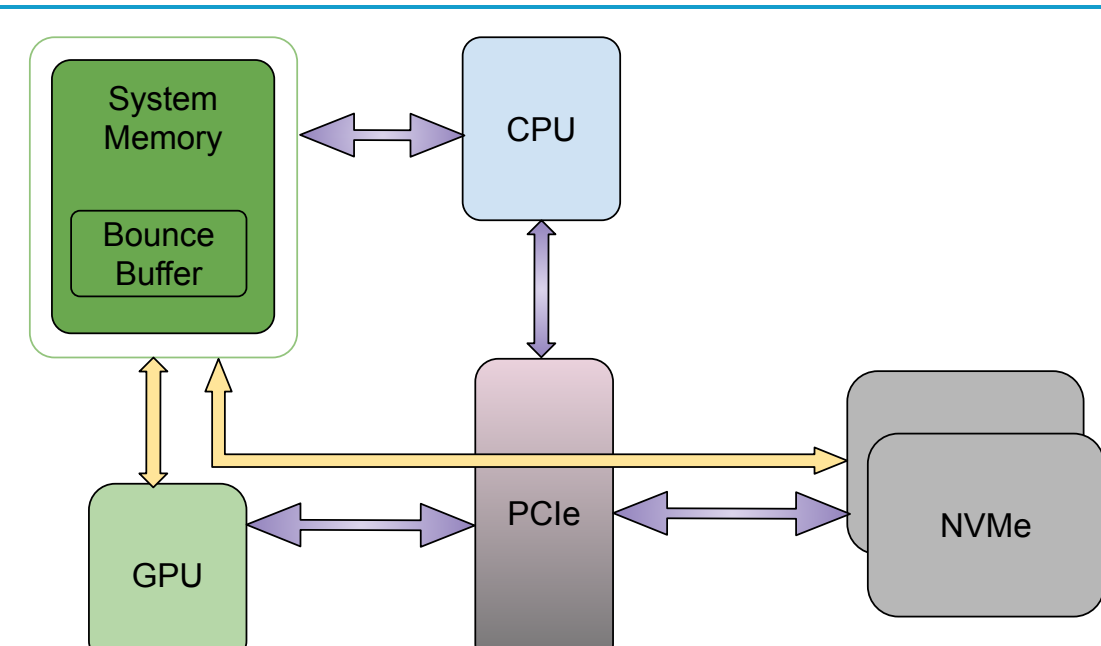


Figure 3: W/o GPUDirect Storage

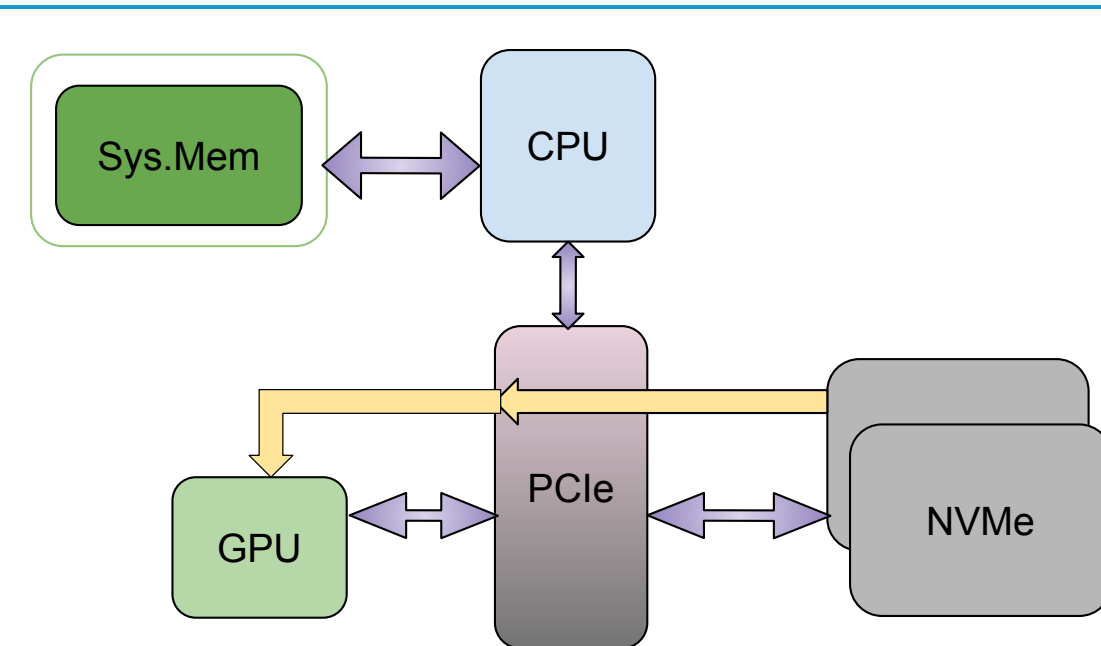


Figure 4: W/ GPUDirect Storage

- W/o GPUDirect (Fig.3), data movement require 3 steps:
 - Load data from storage to DRAM (i. e., Host Mem.)
 - CPU copies data from DRAM into GPU Mem. via `cudaMemcpy()` API
 - GPU kernel accesses the data locally from GPU Mem.
- W/ GPUDirect (Fig.4), data movement only need 2 steps:
 - GPU kernel exposes GPU buffers to Storage devices when applications call `cuFileRead/cuFileWrite` API
 - Data are directly transferred from storage to GPU buffers via DMA mechanism

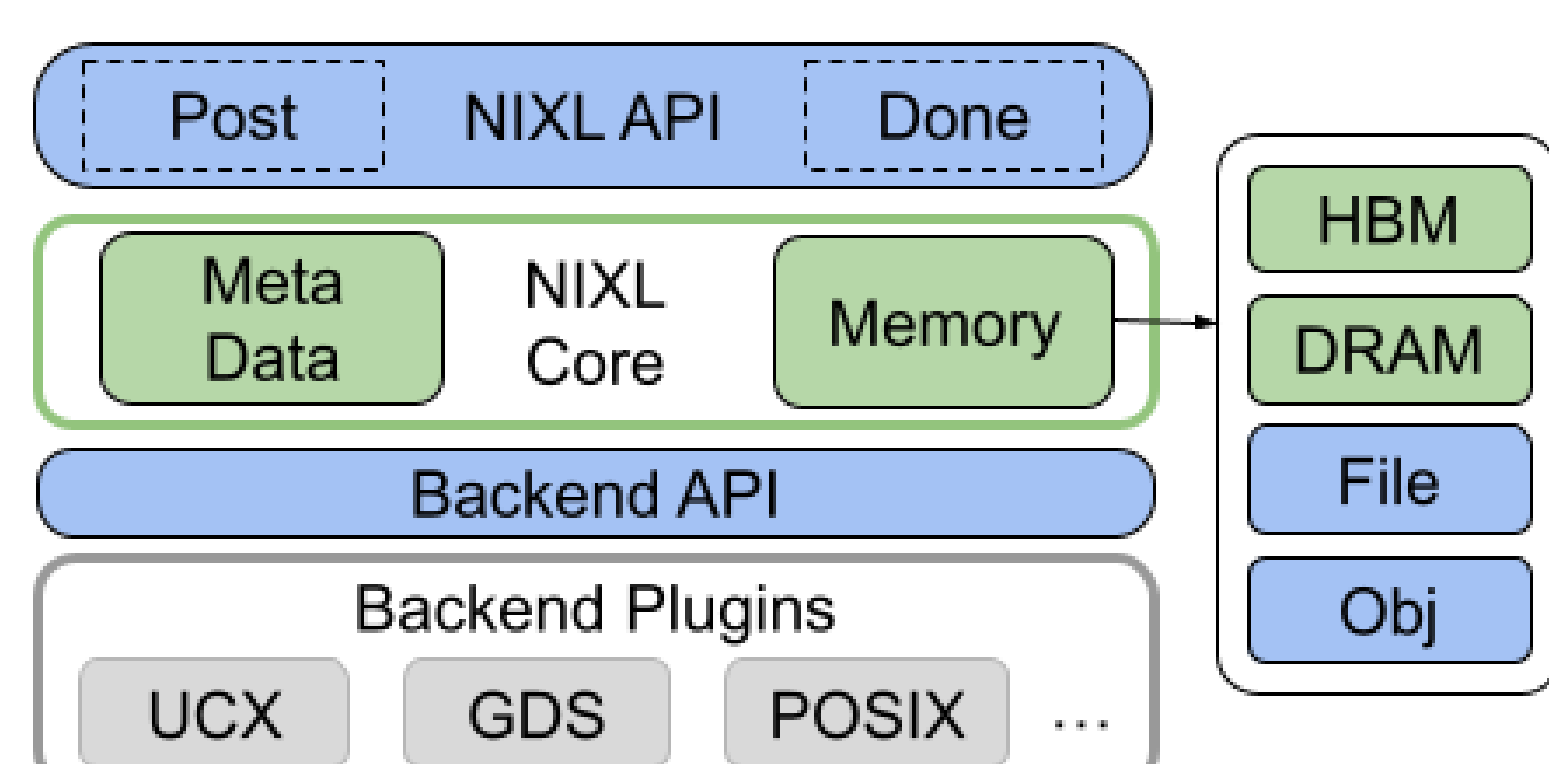


Figure 5: NIXL's Architecture

- NIXL is a **data transfer library**
 - Offer **fast communication for distributed inference workloads** (e.g., efficient KV cache in LLM serving)
 - Provides **backend plugins** to adapt to different data transfer requirements
 - Widely used** by popular inference engines (e.g., Dynamo, vLLM, etc.)

Experiment Setup and Evaluation Approach

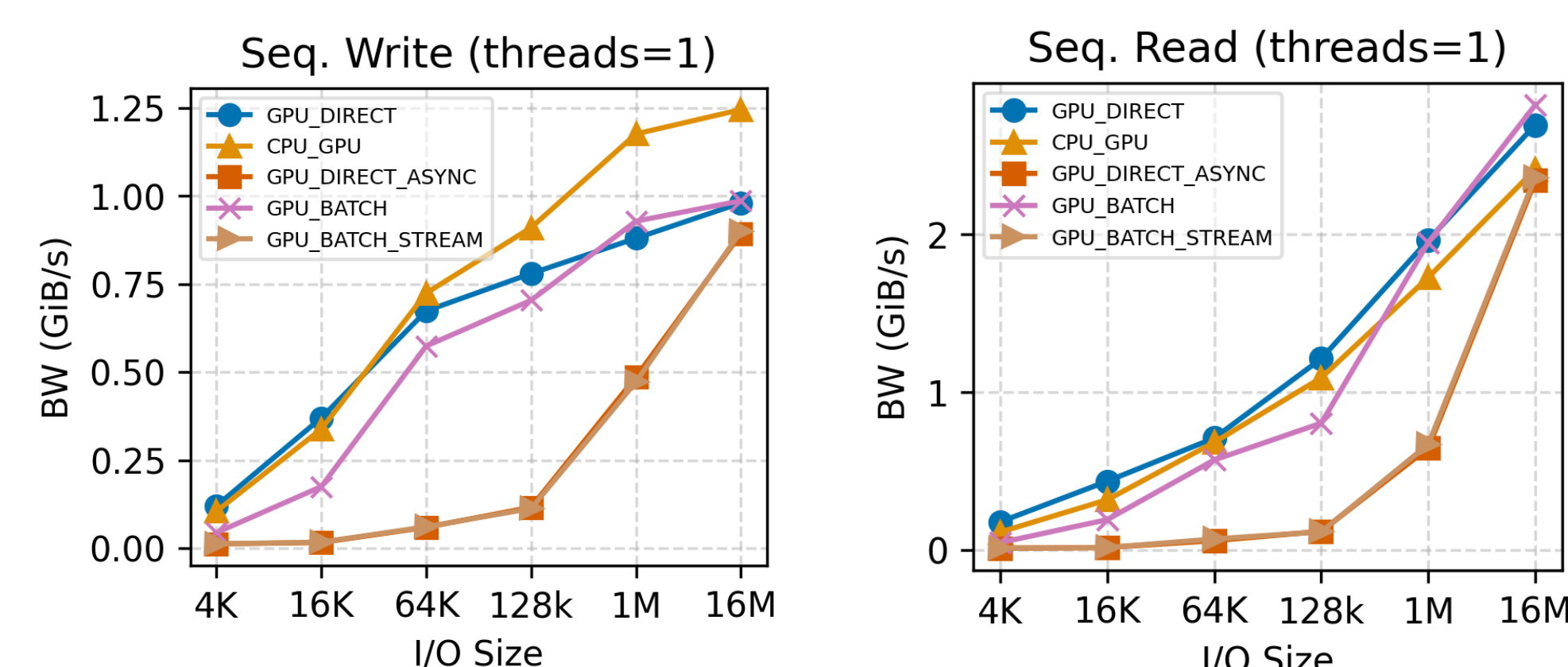
- Platform:** Chameleon cloud, A node with 1 AMD 64-Core CPU, 256GB DRAM, and 1xNVIDIA A100-PCIE-40GB GPU
- Software:** CUDA 12.9; nvidia-fs 2.25 kernel; ext4 file system
- Benchmark Tools:** GDS (via **gdsio v0.4.1**), NIXL (via `nixlbench`)

Xfer_Type	Description
CPU_GPU	Data <-> DRAM <-> GPU (via <code>cudaMemcpy</code>)
GPU_DIRECT	Synchronously move between Data <-> GPU
GPU_DIRECT_ASYNC	Asynchronously move data between Data<-> GPU, allowing overlap I/O and computation
GPU_BATCH	Synchronously batches multiple I/O requests in one batch
GPU_BATCH_STREAM	Asynchronously batches I/O requests via CUDA streams

Acknowledgments

This work is supported in part by the National Science Foundation (NSF), Office of Advanced Cyberinfrastructure (OAC) under Grant 2411318, and the Department of Energy (DOE), Office of Advanced Scientific Computing Research (ASCR) under Grant DE-SC0024593.

Results: GDS W/R via various Transfer Types



- For Write Operations:
 - IO>64KB, GPU_DIRECT slower than CPU_GPU due to **GPU page size limit**
 - Packing overhead** in GPU_BATCH results in slow small IO
 - BATCH_STREAM is internally **Async API + stream-based execution**

- For read, GPU_DIRECT performs best due to **cached metadata**

Results: GDS W/R via various Batch Sizes & Threads

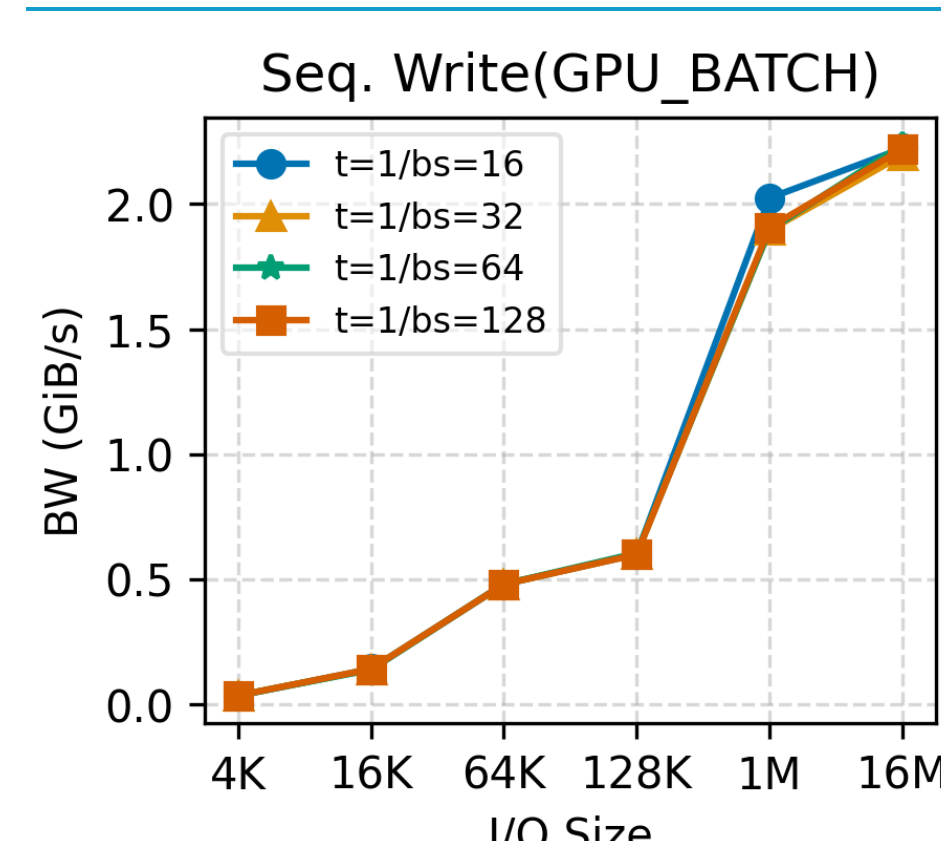


Figure 6: Write via Threads=1

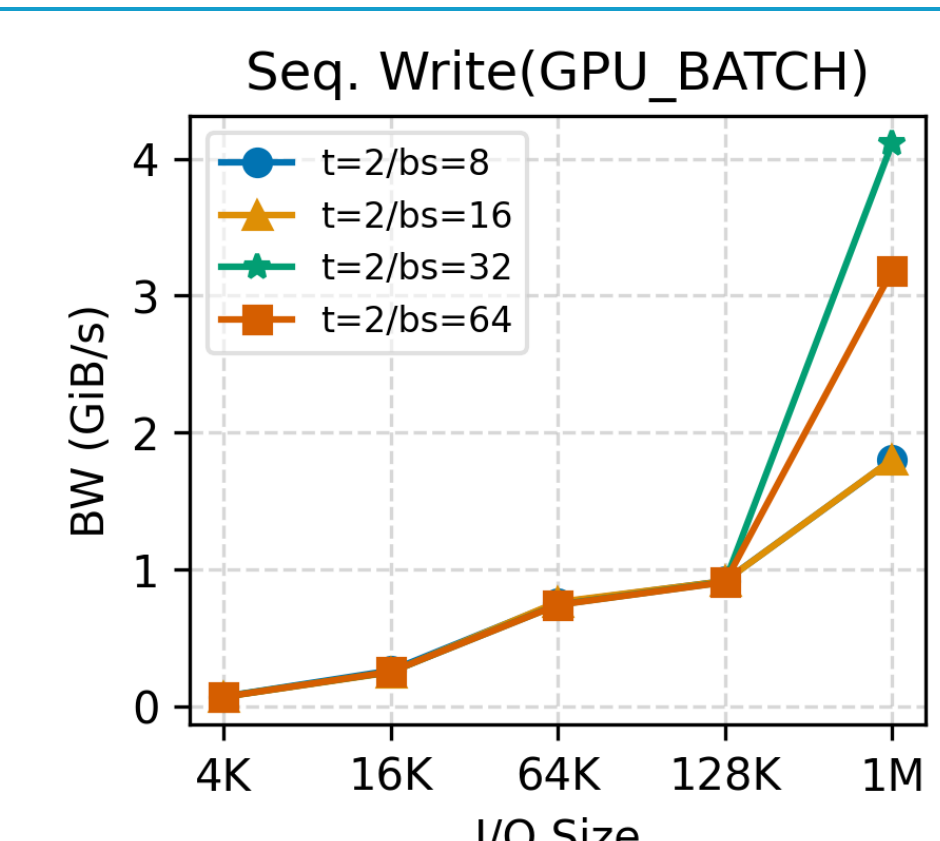


Figure 7: Write via Threads=2

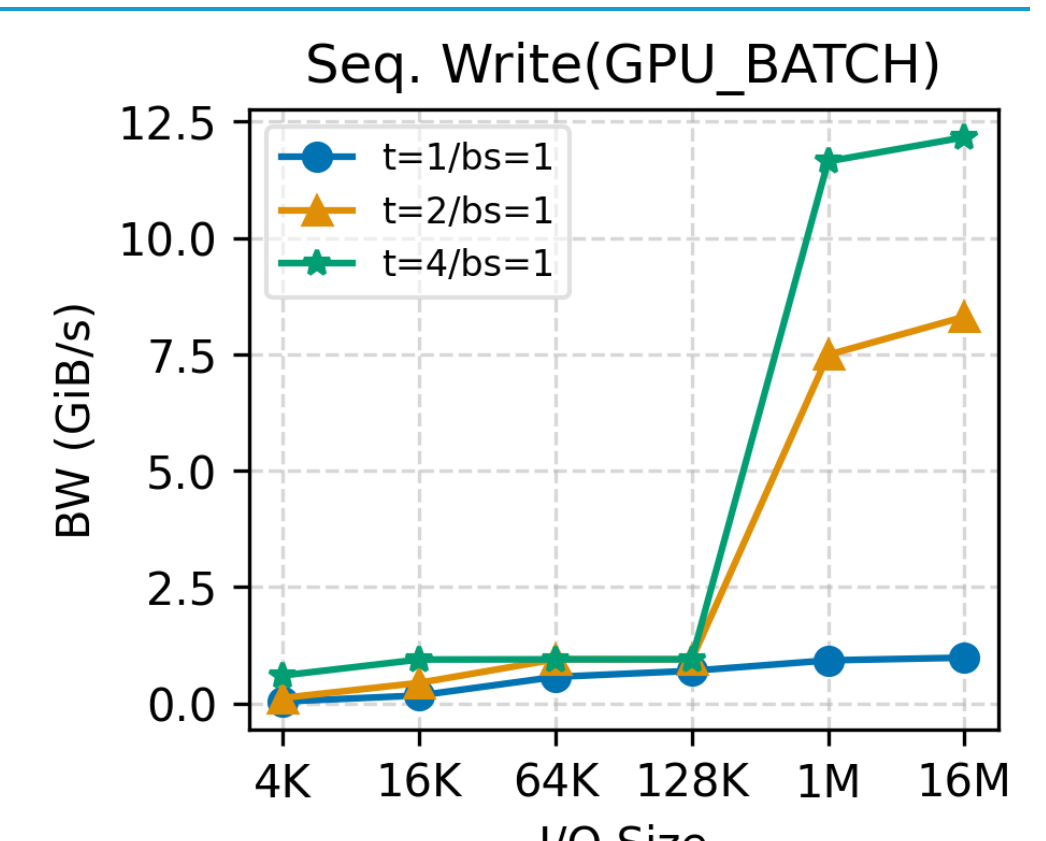


Figure 8: Write, Threads=[1,2,4]

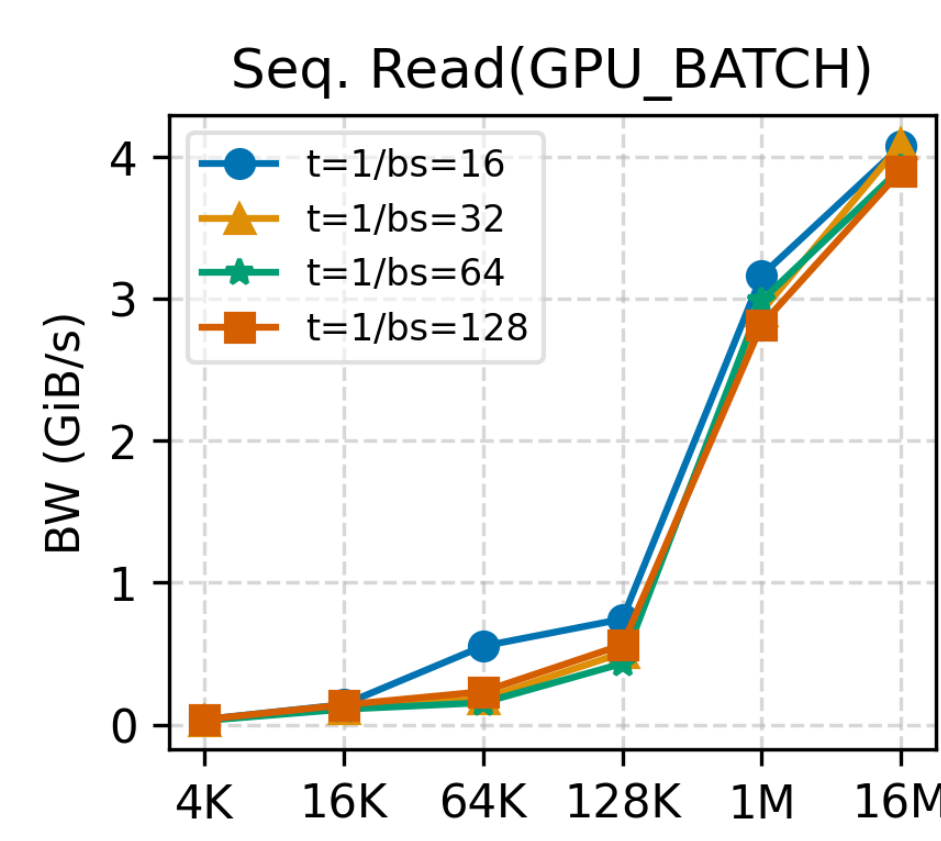


Figure 9: Read with Threads=1

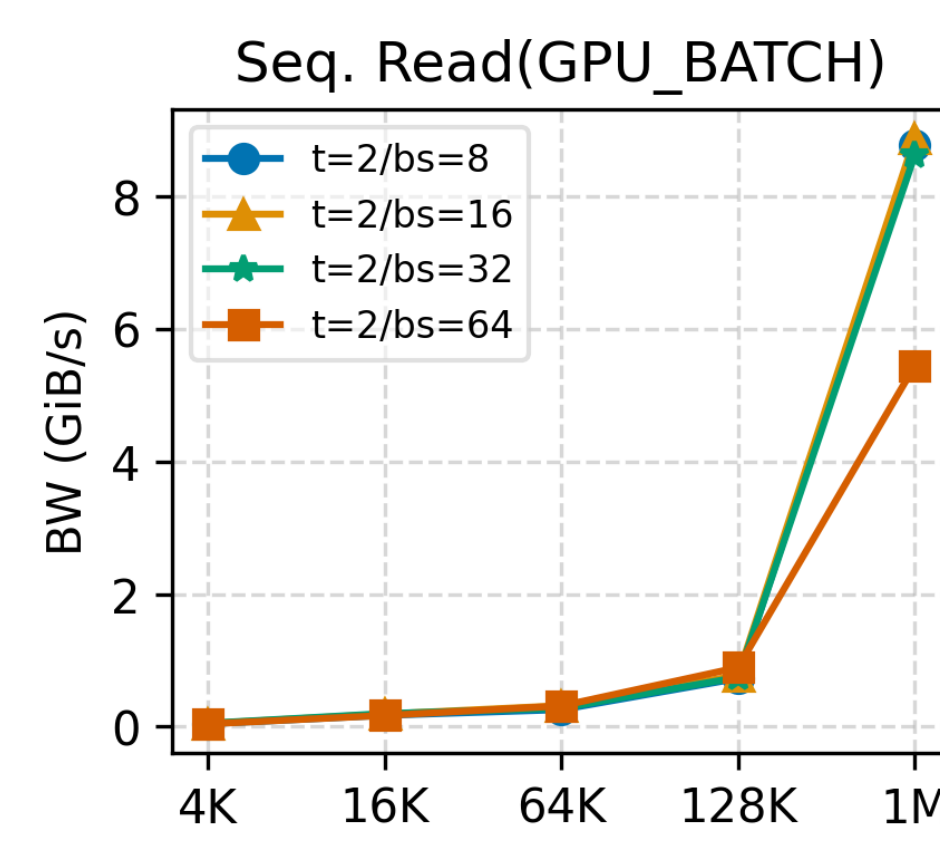


Figure 10: Read with Threads=2

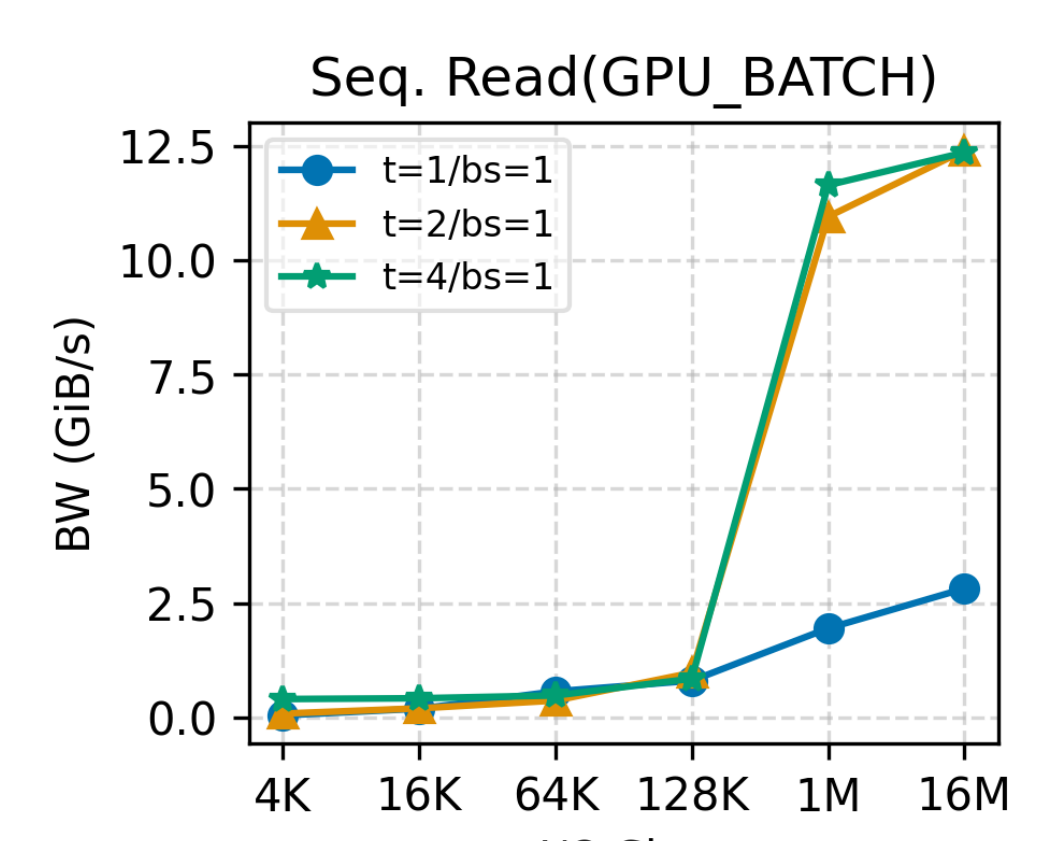


Figure 11: Read, Threads=[1,2,4]

- Multiple threads improve large I/O BW ($\geq 1M$) but minimal impact on small I/O (Fig.8, Fig.11)
 - Batches are split by threads into sub-batches to **parallel process** them, achieving high BW (Fig.7, Fig.10)
 - Each thread **sequentially processes** the sub-batches.

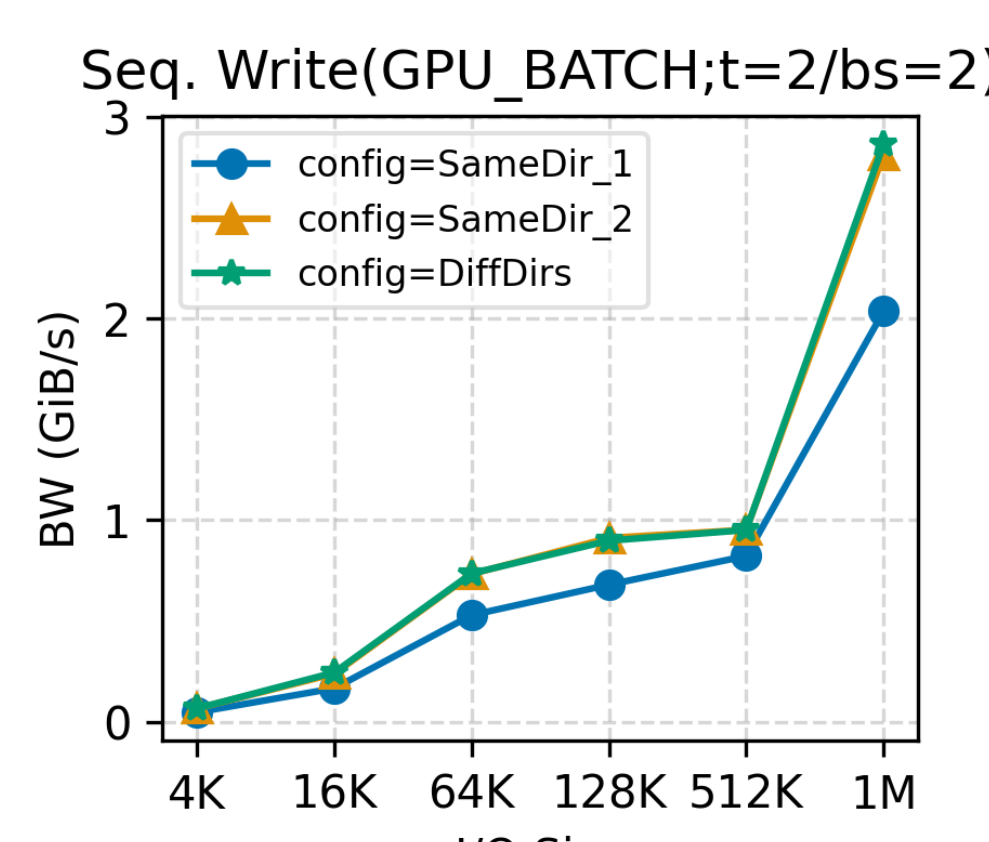


Figure 12: Write configs

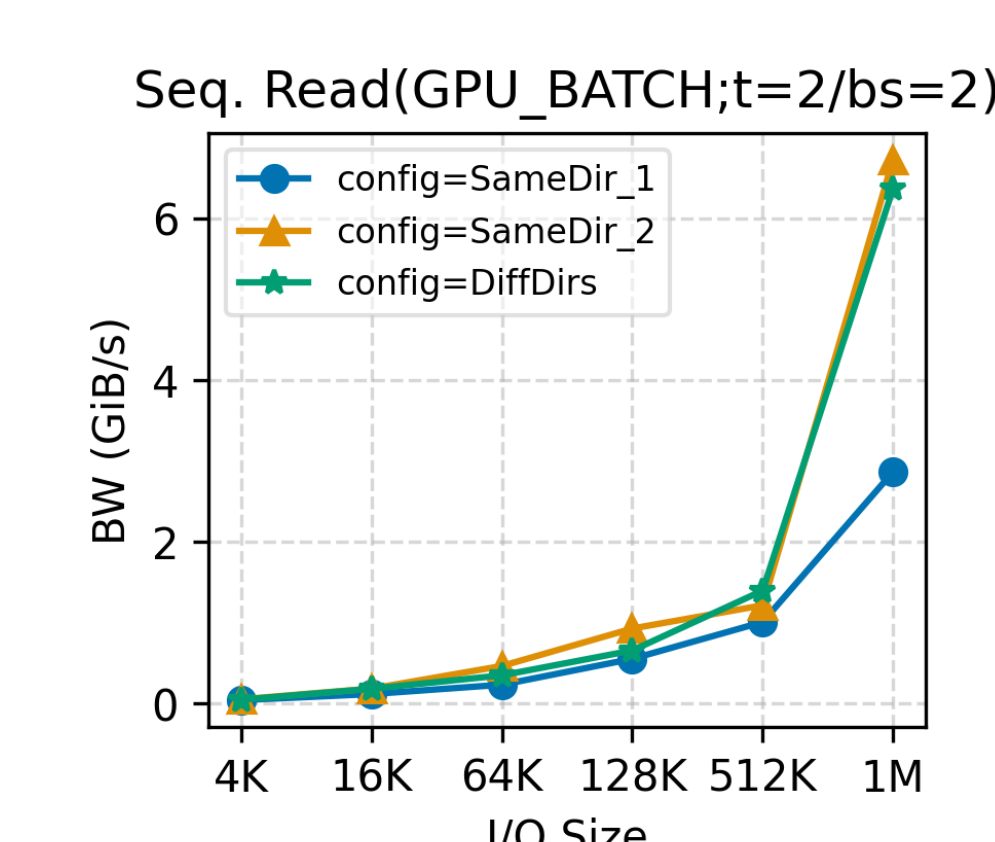


Figure 13: Read configs

For `gdsio` tool, setting **num. of -D parameters = threads** improve write/read BW

Results: NIXL Write/Read with GDS

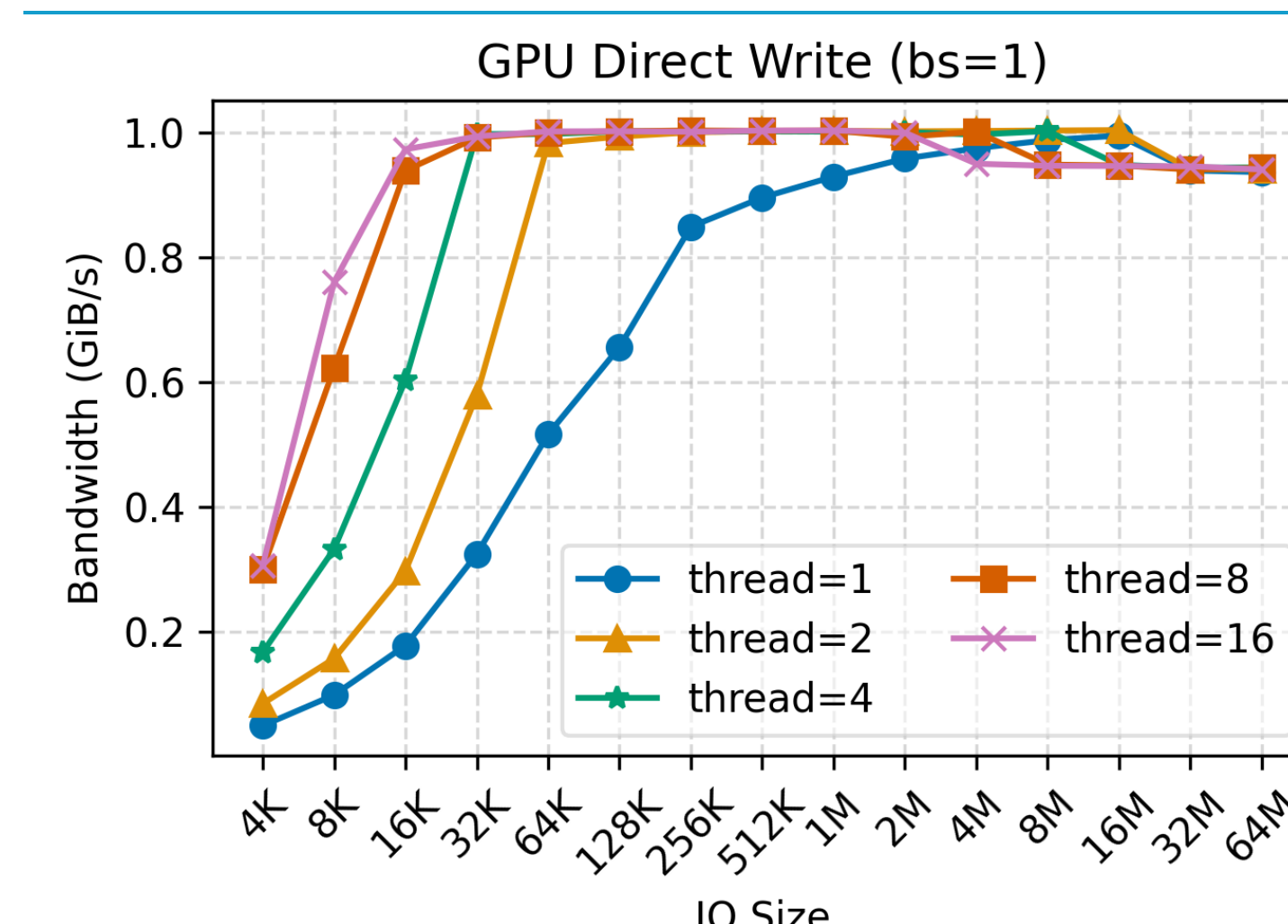


Figure 14: NIXL Write with various threads

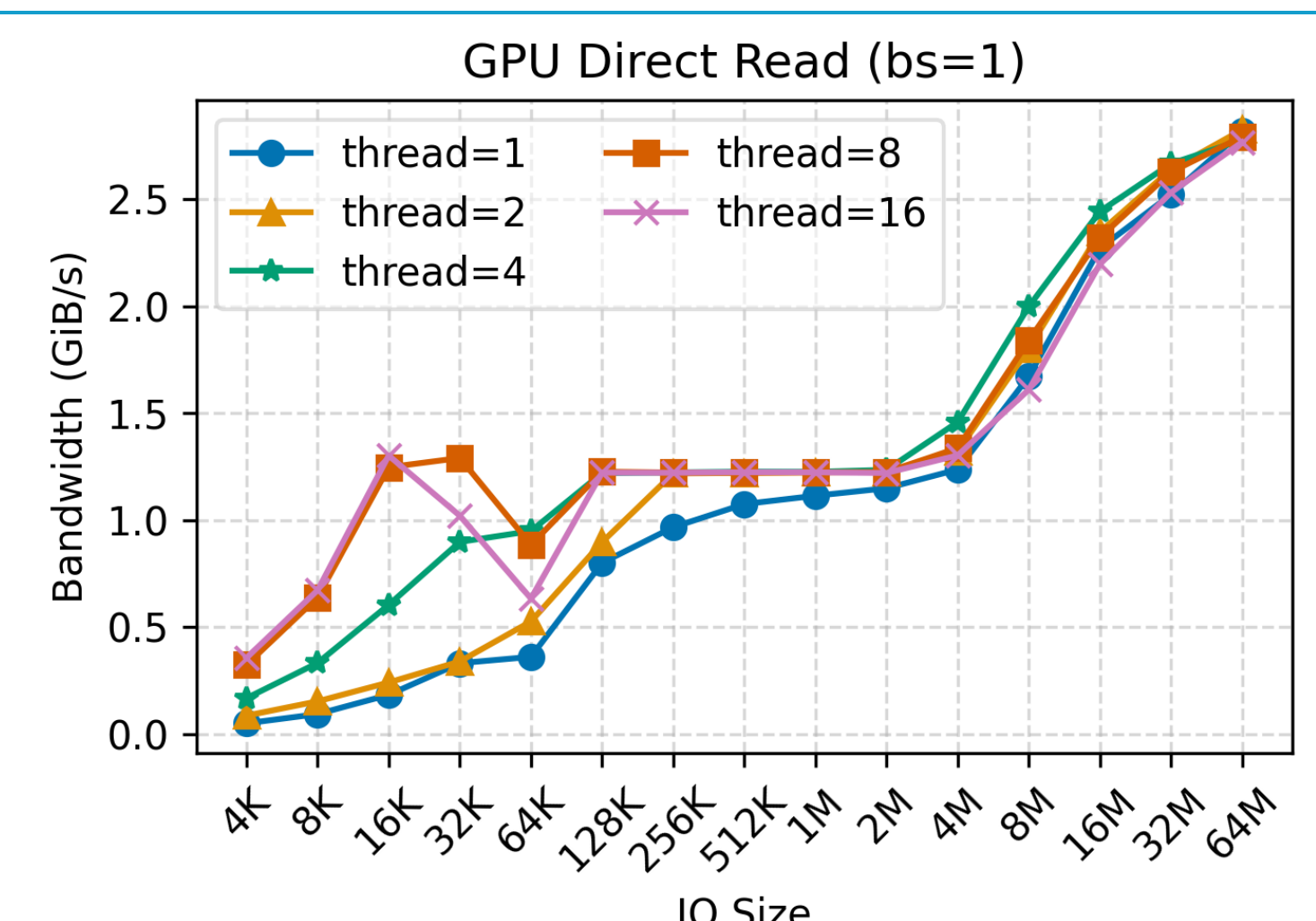


Figure 15: NIXL Read with various threads

- All requests needs to call `cuFileBatchIOSubmit`, adding overhead
- Threads only boost small IO; the 64K read BW drop caused by the GPU's page size limit